Baseline Retrieval vs. Cross-Encoder Re-Ranking

A comparison of two retrieval pipelines over the Magic: The Gathering Comprehensive Rules

Yashwanth B K · 2025201028

LMA Assignment · 20 October 2026

https://github.com/yashwanth-iiit/rag-lma-assignment

1 Setup

I used the *Magic: The Gathering Comprehensive Rules* (effective 7 June 2024) as the document. It is 292 pages of numbered rules, and it is full of the things this assignment is looking for: prohibitions ("can't", "may not"), exceptions ("except", "unless"), and the same words reused in unrelated contexts. The word *counter*, for instance, means both "counter a spell" and "a counter on a permanent", which turned out to be a useful source of keyword traps.

Rather than splitting the text every N characters, I chunked on rule boundaries. The PDF is parsed into individual numbered rules (`701.17a`, `704.5j`, and so on), the table-of-contents pages are dropped, and consecutive rules are packed into chunks of about 700 characters with one rule of overlap. This gave 1,937 chunks, averaging 642 characters and capped at 1,099. The cap matters because the cross-encoder truncates at 512 tokens, and a chunk that gets silently truncated is a chunk the re-ranker cannot score properly.

Both pipelines share the same chunks, the same top-10 candidate list, and the same prompt string. The prompt is defined once as a constant, so the two pipelines cannot drift apart. The LLM was `google/gemini-2.5-flash` through OpenRouter at temperature 0, so any difference between Answer 1 and Answer 2 comes from the retrieved context and not from sampling.

2 Architecture comparison

A bi-encoder puts the query and the chunk through the transformer *separately* and gets two fixed vectors back, which are then compared with cosine similarity. The important consequence is that the chunk vector never depends on the query, so the whole corpus can be embedded once, offline, and reused forever. Answering a query costs one embedding plus a vector lookup.

A cross-encoder concatenates the query and the chunk into a single sequence and runs them through the model *together*. Every query token can attend to every chunk token, so the model is judging the relationship between the two rather than comparing two summaries that were produced in isolation. The output is one relevance score.

That is the whole trade-off. The cross-encoder sees more and scores better, but nothing can be precomputed, because the score depends on the pair. Every candidate needs its own forward pass at query time. The bi-encoder is cheap and cacheable but has to compress a chunk into a single vector before it has any idea what the query will be, and that compression is where the failures in the next section come from.

3 Rank shifts and answer quality

I did not pick the test queries by guessing. I wrote about a hundred candidate questions, found the gold chunk for each by searching the corpus for the rule number that actually answers it, and then recorded where that chunk landed in the baseline top-10 and where it landed after re-ranking. The table below is the subset that made the final cut.

Category	Query (abridged)	Baseline	Re-ranked
Keyword trap	Look at another player's hand?	#8	#1
Keyword trap	Target a creature with protection?	#9	#3
Keyword trap	Tap an already-tapped permanent?	#5	#1
Exception	Both win and lose simultaneously?	#7	#1
Exception	Mana ability while casting a spell?	#8	#2
Exception	Trample blocked by protection?	#5	#2
Negation	Respond to a mana ability?	#8	#1
Negation	Sacrifice a permanent in the graveyard?	#4	#1
Buried chunk	Planeswalker at zero loyalty?	#8	#2
Buried chunk	Creature with lethal damage marked?	#5	#2
Control	What happens when a spell is countered?	#1	#4

Ten of the eleven queries moved the gold chunk from rank 4–10 into the top three. The pattern is consistent: whenever the query shares surface vocabulary with an irrelevant passage, the bi-encoder's single pooled vector gets pulled toward the lexical match. "Look at another player's hand" is about information access, but the top baseline chunk is about hand size, because the word *hand* is doing all the work.

The answer-level differences were more interesting than the rank numbers.

The clearest failure was a misstatement. For "what happens if a player would both win and lose the game at the same time", the baseline put the correct rule (104.3f) at rank 7 and instead surfaced the general draw rules. Answer 1 confidently said *the game is a draw*, citing 104.4a. That is simply wrong: 104.3f says the player *loses*. Answer 2, with 104.3f in the top three, got it right. This is the case I would point to if someone asked why re-ranking is worth the cost, because the baseline answer is not vague or hedged, it is confidently incorrect.

The second was an omission. For "can a player look at another player's hand?", Answer 1 never found 402.3. It reasoned from rule 811.5 (teammates cannot review each other's hands) and rule 720.4 (a player controlling another player can see their hand) and produced a hedged answer about "specific conditions". It is not exactly wrong, but it is not the rule either. Answer 2 quotes 402.3 directly: a player cannot look at another player's hand but may count the cards.

Two queries produced outright refusals. "Can you respond to a mana ability?" and the trample-versus-protection question both returned "the context does not contain the answer" under the baseline, and both were answered correctly after re-ranking. The mana ability case is a good illustration of the negation problem: the bi-encoder ranked chunks that *mention* mana abilities and priority above the chunk that states the prohibition (605.3b), so the model never saw the sentence saying it cannot be responded to.

One case got worse. "What happens when a spell is countered?" starts with the gold chunk at rank 1 and re-ranking pushes it to rank 4. The cross-encoder promoted a chunk about abilities that trigger when a spell is countered, and another about resolution order, above the chunk that actually defines countering. Answer 1 was correct; Answer 2 refused. I think the explanation is that `ms-marco-MiniLM-L-6-v2` is trained on MS MARCO web passages, where a passage that discusses a topic in context often looks more relevant than a terse definitional one. It is optimised for "is this passage relevant", not "is this the authoritative definition". I am keeping this case in the report rather than dropping it, because it shows the method is not a free win.

4 Latency

Both models were warmed up with a dummy call first, then each query was timed 15 times. All measurements are on CPU (12 cores, no GPU).

Stage	Mean (ms)	Std (ms)
Query embedding (bi-encoder)	30.79	9.18
FAISS search (top-10)	1.65	1.01
Baseline retrieval total	**32.44**	**9.94**
Cross-encoder scoring (10 pairs)	708.31	122.70
Re-ranking pipeline total	**740.75**	**119.28**

Building the index took 131 seconds, but that is a one-off cost and is not part of the query path. The cross-encoder accounts for about 96% of the re-ranking pipeline and adds roughly 21.8× the baseline retrieval cost, at about 70.8 ms per pair. The FAISS lookup itself is negligible at 1.65 ms, so the vector search is not the bottleneck at this corpus size.

In absolute terms the overhead is under a second, which is why re-ranking ten candidates is perfectly practical. It is only impractical when you try to do it over the whole corpus, which is the next question.

5 Why not run the cross-encoder over everything?

Because the score depends on the pair, there is nothing to precompute. Running the cross-encoder over the full corpus means N forward passes per query, where N is the number of chunks. Using the measured per-pair cost of 70.8 ms:

Corpus size	Cross-encoder cost per query
1,937 chunks (this corpus)	$\approx$ 137 seconds
100,000 chunks	$\approx$ 2 hours
1,000,000 chunks	$\approx$ 20 hours

And none of it can be cached, because the query changes every time. The bi-encoder does not have this problem: it embeds the corpus once (131 seconds here) and then answers each query in about 32 ms.

This is the entire argument for retrieve-then-rerank. The cheap model cuts N down to $k = 10$, and the expensive model only ever sees those ten candidates. The re-ranking cost then becomes independent of corpus size: it is about 708 ms whether the corpus has two thousand chunks or two million. The cross-encoder is not a replacement for the vector search, it is a second pass that only makes sense after the first pass has done the filtering.

6 Closing note

The one thing I would change if I did this again is the prompt. My first version told the model to say "the provided context does not contain the answer" when it could not find one, and that instruction made it bail out whenever a distractor chunk was present, even when the gold chunk was sitting right there in the context. Removing the escape hatch and asking it to cite the rule number it used fixed most of the false refusals. It is a small reminder that in a RAG system the retrieval and the prompt are not independent, and a prompt that is too cautious can hide a retrieval improvement that is actually working.